

Einführung

Algorithmus

Definition

Ein Algorithmus ist eine **endliche Folge** von Regeln, nach denen sich nach **endlich vielen, eindeutig festgelegten** Schritten die Lösung einer **Klasse von Problemen** ergibt.

Eigenschaften

- **Finitheit**
 - Wir können den Algorithmus mit endlichen Ressourcen beschreiben
- **Terminiertheit**
 - Der Algorithmus endet nach endlicher Zeit
- **Determiniertheit**
 - Es gibt keinen Interpretationsspielraum
- **Allgemeingültigkeit**
 - Lösung wird für unterschiedliche Eingaben errechnet
- **Korrektheit**
 - Ein Algorithmus ist korrekt, wenn er die Aufgabe richtig löst
- **Effizienz**
 - Ein Algorithmus ist effizient, wenn er vergleichsweise wenige Schritte/Operationen und wenig Speicher benötigt

Datenstruktur

Die Art und Weise, die die von einem Algorithmus bearbeiteten Daten organisiert und vorgehalten werden.

Algorithmen und Datenstrukturen bedingen sich gegenseitig, d.h. für die Durchführung eines Algorithmus ist es i.d.R. notwendig, dass die Daten in einer bestimmten Datenstruktur vorliegen.

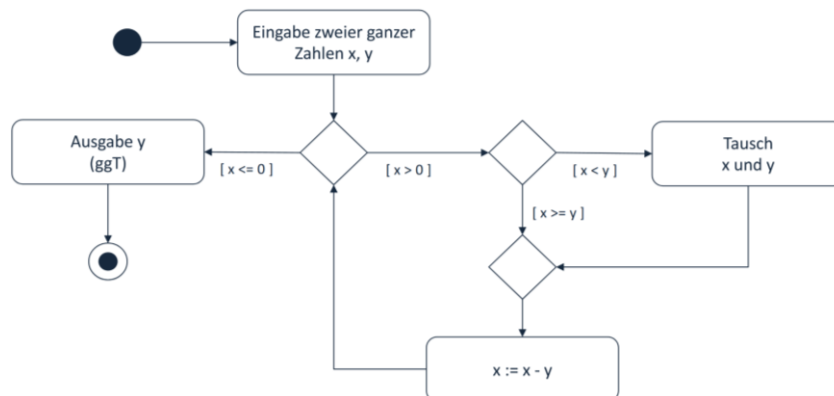
Manchmal ist das Erzeugen der Datenstruktur aufwändiger als die anschließende Durchführung des Algorithmus.

Darstellung eines Algorithmus

Informelle Darstellung

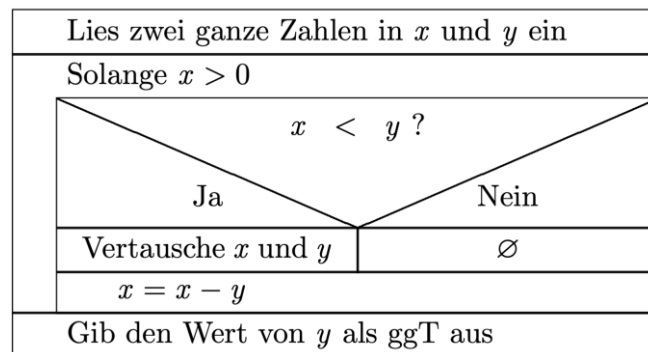
1. Lies zwei ganze Zahlen in die Variablen ,x' und ,y' ein!
2. Solange der Wert von ,x' größer als 0 ist, wiederhole die Schritte 2.1 und 2.2
 - 2.1. Wenn der Wert von ,x' kleiner als ,y', dann vertausche die Inhalte von ,x' und ,y'!
 - 2.2. Weise ,x' die Differenz von ,x' minus ,y' zu!
3. Gib den Wert von ,y' als ggT aus!

UML-Aktivitätsdiagramm



Nassi & Shneiderman Struktogramm (DIN 66261)

Euklid-Algorithmus



Pseudocode

Euklid-Algorithmus

Ensure: $x, y \in \mathbb{N}$

- ```

1: while $x > 0$ do
2: if $x < y$ then
3: $temp \leftarrow x$
4: $x \leftarrow y$
5: $y \leftarrow temp$
6: end if
7: $x \leftarrow x - y$
8: end while
9: return y

```

▷  $y$  enthält den gesuchten ggT

# Landau Symbole

## Beschreibung

Mit Landau-Symbolen lassen sich Algorithmen hinsichtlich ihres Bedarfs an Zeit und/oder Speicher bezüglich eines bestimmten Maschinenmodells analysieren.

Man ermittelt dabei die Schranken, in denen sich Laufzeit und/oder Speicherplatzbedarf hält, wenn man die Problemgröße  $n$  vergrößert.

Mit den Landau-Symbolen beschreibt man eine Menge von Funktionen, deren Wachstumsraten vergleichbare Wachstumsraten haben wie die Funktion  $g$ .

| Notation          | Bedeutung                                       | Sprechweise  |
|-------------------|-------------------------------------------------|--------------|
| $f \in O(g)$      | $f$ wächst <b>höchstens</b> so schnell wie $g$  | „groß O“     |
| $f \in \Omega(g)$ | $f$ wächst <b>mindestens</b> so schnell wie $g$ | „groß Omega“ |
| $f \in \Theta(g)$ | $f$ wächst <b>genauso</b> schnell wie $g$       | „groß Theta“ |

$$\Theta(g) \subseteq O(g)$$

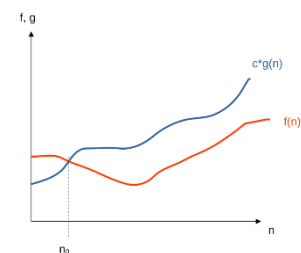
$$\Theta(g) \subseteq \Omega(g)$$

$$\Theta(g) = O(g) \cap \Omega(g)$$

## O-Notation

Es gibt eine **obere Schranke**, die das Wachstum ab einer bestimmten Größe von  $n$  nicht überschreitet.

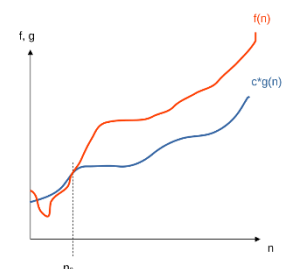
$$O(g(n)) = f(n): \text{es gibt positive } c \text{ und } n_0, \\ \text{dass für alle } n \geq n_0 \text{ gilt:} \\ 0 \leq f(n) \leq c \cdot g(n) \\ \rightarrow f \in O(g)$$



## Ω-Notation

Es gibt eine **untere Schranke**, die das Wachstum ab einer bestimmten Größe von  $n$  nicht unterschreitet. (verwendet zur Beschreibung der Best-Case-Laufzeit)

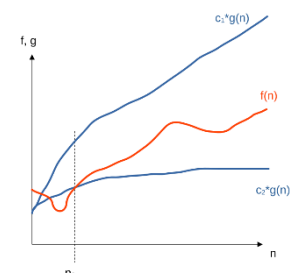
$$\Omega(g(n)) = f(n): \text{es gibt positive } c \text{ und } n_0, \\ \text{dass für alle } n \geq n_0 \text{ gilt:} \\ 0 \leq c \cdot g(n) \leq f(n) \\ \rightarrow f \in \Omega(g)$$



## Θ-Notation

Es gibt eine **obere Schranke** und eine **untere Schranke**, welche das Wachstum ab einer bestimmten Größe von  $n$  eingrenzen.

$$\Theta(g(n)) = f(n): \text{es gibt positive } c_1, c_2 \text{ und } n_0, \\ \text{dass für alle } n \geq n_0 \text{ gilt:} \\ 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \\ \rightarrow f \in \Theta(g)$$



## Master-Theorem

Das Master-Theorem (auch Hauptsatz der Laufzeitfunktionen) ist ein mathematisches Hilfsmittel der Informatik. Es dient der schnellen Bestimmung der asymptotischen Laufzeit (O-Notation) von rekursiven „Teile-und-Herrsche“ (Divide-and-Conquer)-Algorithmen, ohne die Rekursion explizit auflösen zu müssen.

### Allgemeine Rekurrenzformel

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

$n$ : Größe des Eingabeproblems

$a$ : Anzahl der Teilprobleme  $\geq 1$

$b$ : Verkleinerungsfaktor  $> 1$

$f(n)$ : Kosten außerhalb der Rekursion

### Checkliste

1.  $a, b$  und  $f(n)$  identifizieren
2.  $n^{\log_b a}$  berechnen
3.  $f(n)$  mit  $n^{\log_b a}$  vergleichen
4. Passenden Fall anwenden

### MAXFOLGE3

$a = 2, b = 2$  und  $f(n) = n$

$n^{\log_b a} = n$

Beide gleich  $\rightarrow$  Fall 2

$\Theta(f(n) \cdot \lg n)$

### drei Fälle

Das Master-Theorem vergleicht den Aufwand für die Rekursion dem Aufwand für das Zusammenfügen  $f(n)$ . Die kritische Größe ist dabei  $n^{\log_b a}$ .

#### Fall 1: Rekursion dominiert

$f(n)$  wächst langsamer als  $n^{\log_b a} \rightarrow T(n) \in \Theta(n^{\log_b a})$

#### Fall 2: Arbeit ist gleichmäßig verteilt

$f(n)$  wächst so schnell wie  $n^{\log_b a} \rightarrow T(n) \in \Theta(f(n) \cdot \log_b n)$  (auf jeder Ebene  $f(n)$ )

#### Fall 3: Teilen/Zusammenführen dominiert

$f(n)$  wächst schneller als  $n^{\log_b a} \rightarrow T(n) \in \Theta(f(n))$

## Sortiervverfahren

Ein Sortieralgorithmus ist korrekt, wenn er terminiert und am Ende die Sortiertheitsbedingung erfüllt.

Sortiertheitsbedingung:  $\forall i \in 0, \dots, n-2 : Z[i] \leq Z[i+1]$

### Korrektheitsbeweis durch Induktionsbeweis

1. Welche Aussage gilt vor dem ersten Durchlauf?
2. Welche Aussage bleibt nach jedem Durchlauf erhalten (Schleifeninvariante)?
3. Welche Aussage impliziert bei Terminierung die Sortiertheitsbedingung?

### Stabilität

Ein Sortieralgorithmus gilt als stabil, wenn er bei gleichen Elementen die zuvor vorliegende Reihenfolge der Elemente relativ zueinander beibehält.

### In-Place

Ein Sortieralgorithmus heißt in-place, wenn er nur eine konstante Menge an zusätzlichem Speicher benötigt, unabhängig von der Eingabegröße.

### Kostenmaß

Für Sortiervverfahren wird zum Vergleichen die Anzahl der Vergleiche genutzt.

Sortierproblem ist „ $\Omega(n \log n)$ -hart“

### Zusammenfassung

| Sortiervverfahren | Worst Case    | Average Case  | Best Case     | Stabil? | In-Place? |
|-------------------|---------------|---------------|---------------|---------|-----------|
| Insert-Sort       | $O(n^2)$      | $O(n^2)$      | $O(n)$        | Ja      | Ja        |
| Select-Sort       | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | Nein    | Ja        |
| Bubble-Sort       | $O(n^2)$      | $O(n^2)$      | $O(n)$        | Ja      | Ja        |
| Quicksort         | $O(n^2)$      | $O(n \log n)$ | $O(n \log n)$ | Nein    | Nein      |
| Heapsort          | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | Nein    | Ja        |
| Merge-Sort        | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | Ja      | Nein      |
| Counting-Sort     | $O(n)$        | $O(n)$        | $O(n)$        | Ja      | Nein      |

**Hinweis:**  $\log n = \frac{\lg n}{\lg 10}$

## Algorithmus: Maximale Abschnittsumme

Finden der maximalen Abschnittssumme für eine Abfolge von  $n$  ganzen Zahlen. Die Zahlen liegen in einem Feld  $Z$  mit den Indizes  $0 \dots n - 1$  im Speicher vor.

### 1. Lösungsansatz *MAXFOLGE1(Z)*

Betrachten und berechnen aller Teilfolgen und Auswahl des Maximums.

Komplexität:  $O(n^3) \rightarrow$  **kubischer Algorithmus.**

---

#### MAXFOLGE1(Z)

---

```

1: $n \leftarrow \text{length}(Z)$
2: $max \leftarrow -\infty$
3: for $i \leftarrow 0$ to $n - 1$ do
4: for $j \leftarrow i$ to $n - 1$ do
5: $sum \leftarrow 0$
6: for $k \leftarrow i$ to j do
7: $sum \leftarrow sum + Z[k]$
8: end for
9: if $sum > max$ then
10: $max \leftarrow sum$
11: $links \leftarrow i$
12: $rechts \leftarrow j$
13: end if
14: end for
15: end for
16: return $(max, links, rechts)$
```

---

### 2. Lösungsansatz *MAXFOLGE2(Z)*

Verbesserung zu *MAXFOLGE1(Z)*.

Aufsummieren von jedem möglichen Startpunkt.

Komplexität:  $O(n^2) \rightarrow$  **quadratischer Algorithmus.**

---

#### MAXFOLGE2(Z)

---

```

1: $n \leftarrow \text{length}(Z)$
2: $max \leftarrow -\infty$
3: for $i \leftarrow 0$ to $n - 1$ do
4: $sum \leftarrow 0$
5: for $j \leftarrow i$ to $n - 1$ do
6: $sum \leftarrow sum + Z[j]$
7: if $sum > max$ then
8: $max \leftarrow sum$
9: $links \leftarrow i$
10: $rechts \leftarrow j$
11: end if
12: end for
13: end for
14: return $(max, links, rechts)$
```

---

### 3. Lösungsansatz „Teile-und-Herrsche“

Teilt das Feld  $Z[l \dots r]$  in der Mitte  $m$ ,

dass **zwei Teilabschnitte**  $Z[l \dots m]$  und  $Z[m + 1 \dots r]$  entstehen.

Der gesuchte Abschnitt mit der maximalen Summe kann sich nun ...

1. im Abschnitt  $Z[l \dots m]$  (links) befinden  $\rightarrow \text{MAXFOLGE3}(Z, l, m)$
2. im Abschnitt  $Z[m + 1 \dots r]$  (rechts) befinden  $\rightarrow \text{MAXFOLGE3}(Z, m + 1, r)$
3. über beide Abschnitte (zwischen) erstrecken  $\rightarrow \text{FINDE-ZWISCHEN}(Z, l, m, r)$

Komplexität:  $O(n \lg n)$

---

#### FINDE-ZWISCHEN( $Z, l, m, r$ )

---

```

1: linksMax $\leftarrow -\infty$
2: sum $\leftarrow 0$
3: for $i \leftarrow m$ downto l do
4: sum $\leftarrow \text{sum} + Z[i]$
5: if sum > linksMax then
6: linksMax $\leftarrow \text{sum}$
7: links $\leftarrow i$
8: end if
9: end for
10: rechtsMax $\leftarrow -\infty$
11: sum $\leftarrow 0$
12: for $i \leftarrow m + 1$ to r do
13: sum $\leftarrow \text{sum} + Z[i]$
14: if sum > rechtsMax then
15: rechtsMax $\leftarrow \text{sum}$
16: rechts $\leftarrow i$
17: end if
18: end for
19: return (linksMax + rechtsMax, links, rechts)
```

---



---

#### MAXFOLGE3( $Z, l, r$ )

---

```

1: if $l = r$ then $\triangleright$ Abbruchbedingung für die Rekursion
2: return ($Z[l], l, r$)
3: end if
4: $m \leftarrow (l + r) / 2$ $\triangleright$ Ganzzahlige Division
5: (linksMax, linksL, linksR) $\leftarrow \text{MAXFOLGE3}(Z, l, m)$ $\triangleright$ 1. Fall
6: (rechtsMax, rechtsL, rechtsR) $\leftarrow \text{MAXFOLGE3}(Z, m + 1, r)$ $\triangleright$ 2. Fall
7: (zwiMax, zwiL, zwiR) $\leftarrow \text{FINDE-ZWISCHEN}(Z, l, m, r)$ $\triangleright$ 3. Fall
8: if linksMax $\geq$ rechtsMax and linksMax $\geq$ zwiMax then
9: return (linksMax, linksL, linksR)
10: end if
11: if rechtsMax $\geq$ linksMax and rechtsMax $\geq$ zwiMax then
12: return (rechtsMax, rechtsL, rechtsR)
13: end if
14: return (zwiMax, zwiL, zwiR)
```

---

#### 4. Lösungsansatz *MAXFOLGE4(Z)*

Die Folge wird einmal von links nach rechts durchlaufen und aufsummiert.

Die maximale Summe wird abgespeichert.

Komplexität:  $O(n) \rightarrow$  **linearer Algorithmus**.

---

**MAXFOLGE4(*Z*)**

---

```
1: $n \leftarrow \text{length}(Z)$
2: $max \leftarrow -\infty$
3: $aktSum \leftarrow 0$
4: $aktLinks \leftarrow 0$
5: for $i \leftarrow 0$ to $n - 1$ do
6: $aktSum \leftarrow aktSum + Z[i]$
7: if $aktSum > max$ then
8: $max \leftarrow aktSum$
9: $links \leftarrow aktLinks$
10: $rechts \leftarrow i$
11: end if
12: if $aktSum < 0$ then
13: $aktSum \leftarrow 0$
14: $aktLinks \leftarrow i + 1$
15: end if
16: end for
17: return $(max, links, rechts)$
```

---



## Algorithmus: Bubble-Sort

Feld wird durchlaufen.

Die nebeneinander liegenden Speicherzellen werden verglichen.

Stimmt die Reihenfolge nicht, werden die Inhalte vertauscht.

Beim ersten Durchgang wandert das kleinste Element an erste Stelle.

Der zweite Durchgang muss nur noch das Feld ab der zweiten Stelle bearbeiten.

Komplexität:  $O(n^2)$

---

### BUBBLE-SORT( $Z$ )

---

```

1: $n \leftarrow \text{length}(Z)$
2: for $i \leftarrow 0$ to $n - 2$ do
3: for $j \leftarrow n - 1$ downto $i + 1$ do
4: if $Z[j - 1] > Z[j]$ then
5: $\text{temp} \leftarrow Z[j]$
6: $Z[j] \leftarrow Z[j - 1]$
7: $Z[j - 1] \leftarrow \text{temp}$
8: end if
9: end for
10: end for

```

---

Durchlaufen: hinten  $\rightarrow$  vorne

---

### BUBBLE-SORT2( $Z$ )

---

```

1: $n \leftarrow \text{length}(Z)$
2: repeat
3: $\text{sortiert} \leftarrow \text{true}$
4: for $i \leftarrow 0$ to $n - 2$ do
5: if $Z[i] > Z[i + 1]$ then
6: $\text{sortiert} \leftarrow \text{false}$
7: $\text{temp} \leftarrow Z[i]$
8: $Z[i] \leftarrow Z[i + 1]$
9: $Z[i + 1] \leftarrow \text{temp}$
10: end if
11: end for
12: $n \leftarrow n - 1$
13: until $\text{sortiert} = \text{true}$ or $n \leq 1$

```

---

Durchlaufen: vorne  $\rightarrow$  hinten

## Algorithmus: Select-Sort

Kleinstes Element wird ermittelt und an 1. Stelle gesetzt.

Dann wird das zweitkleinste Element und an 2. Stelle gesetzt usw.

Für jede Position  $i$  wird das Feld ab der Position  $i + 1$  durchsucht und ggf. das gefundene kleinste Element an die Position  $i$  getauscht.

Komplexität:  $O(n^2)$

---

**SELECT-SORT( $Z$ )**

---

```
1: $n \leftarrow \text{length}(Z)$
2: for $i \leftarrow 0$ to $n - 1$ do
3: $\text{min} \leftarrow i$
4: for $j \leftarrow i + 1$ to $n - 1$ do
5: if $Z[j] < Z[\text{min}]$ then
6: $\text{min} \leftarrow j$
7: end if
8: end for
9: $\text{temp} \leftarrow Z[i]$
10: $Z[i] \leftarrow Z[\text{min}]$
11: $Z[\text{min}] \leftarrow \text{temp}$
12: end for
```

---

## Algorithmus: Insert-Sort

Insertion-Sort geht die Liste von links nach rechts durch und behandelt jedes Element einzeln.

Dabei wird das aktuelle Element so weit nach links geschoben, bis es an der richtigen Stelle im bereits sortierten Teil steht.

Am Ende ist die gesamte Liste sortiert, weil jedes Element genau dort eingefügt wurde, wo es hingehört.

Im Worst Case muss jedes betrachtete Element an den Anfang geschoben werden, woraus  $\frac{n \cdot (n-1)}{2}$  Vergleiche entstehen.

Komplexität:  $O(n^2)$

---

### INSERT-SORT(*Z*)

---

```

1: $n \leftarrow \text{length}(Z)$
2: for $i \leftarrow 1$ to $n - 1$ do
3: $elem \leftarrow Z[i]$
4: $j \leftarrow i$
5: while $j > 0$ and $Z[j - 1] > elem$ do
6: $Z[j] \leftarrow Z[j - 1]$
7: $j \leftarrow j - 1$
8: end while
9: $Z[j] \leftarrow elem$
10: end for

```

---

Vor Sortierung

44 39 29 58 27 65 1 79 46 10 62 81

|         |     |    |    |    |    |    |    |    |    |    |    |    |    |
|---------|-----|----|----|----|----|----|----|----|----|----|----|----|----|
| -----   |     |    |    |    |    |    |    |    |    |    |    |    |    |
| Schritt | 1:  | 39 | 44 | 29 | 58 | 27 | 65 | 1  | 79 | 46 | 10 | 62 | 81 |
| Schritt | 2:  | 29 | 39 | 44 | 58 | 27 | 65 | 1  | 79 | 46 | 10 | 62 | 81 |
| Schritt | 3:  | 29 | 39 | 44 | 58 | 27 | 65 | 1  | 79 | 46 | 10 | 62 | 81 |
| Schritt | 4:  | 27 | 29 | 39 | 44 | 58 | 65 | 1  | 79 | 46 | 10 | 62 | 81 |
| Schritt | 5:  | 27 | 29 | 39 | 44 | 58 | 65 | 1  | 79 | 46 | 10 | 62 | 81 |
| Schritt | 6:  | 1  | 27 | 29 | 39 | 44 | 58 | 65 | 79 | 46 | 10 | 62 | 81 |
| Schritt | 7:  | 1  | 27 | 29 | 39 | 44 | 58 | 65 | 79 | 46 | 10 | 62 | 81 |
| Schritt | 8:  | 1  | 27 | 29 | 39 | 44 | 46 | 58 | 65 | 79 | 10 | 62 | 81 |
| Schritt | 9:  | 1  | 10 | 27 | 29 | 39 | 44 | 46 | 58 | 65 | 79 | 62 | 81 |
| Schritt | 10: | 1  | 10 | 27 | 29 | 39 | 44 | 46 | 58 | 62 | 65 | 79 | 81 |
| Schritt | 11: | 1  | 10 | 27 | 29 | 39 | 44 | 46 | 58 | 62 | 65 | 79 | 81 |

## Algorithmus: Merge-Sort

Merge-Sort teilt die Eingabeliste rekursiv in immer kleinere Hälften, bis nur noch einzelne Elemente übrig sind.

Diese werden anschließend paarweise wieder zu sortierten Teillisten zusammengeführt.

Durch dieses systematische Teilen und Mergen entsteht am Ende eine vollständig sortierte Liste.

Komplexität:  $O(n \log n)$

MERGE-SORT(A, l, r)

```

1: if l = r then
2: return (A[l], l, r)
3: end if
4: m ← ⌊(l + r) / 2⌋
5: (linksSum, linksL, linksR) ← MERGE-SORT(A, l, m)
6: (rechtsSum, rechtsL, rechtsR) ← MERGE-SORT(A, m + 1, r)
7: (crossSum, crossL, crossR) ← MERGE(A, l, m, r)
8: if linksSum ≥ rechtsSum and linksSum ≥ crossSum then
9: return (linksSum, linksL, linksR)
10: else if rechtsSum ≥ linksSum and rechtsSum ≥ crossSum then
11: return (rechtsSum, rechtsL, rechtsR)
12: else
13: return (crossSum, crossL, crossR)
14: end if

```

MERGE(A, l, m, r)

```

1: i ← l
2: j ← m + 1
3: B ← leere Liste
4: while i ≤ m and j ≤ r do
5: if A[i] ≤ A[j] then
6: B.append(A[i])
7: i ← i + 1
8: else
9: B.append(A[j])
10: j ← j + 1
11: end if
12: end while
13: while i ≤ m do
14: B.append(A[i])
15: i ← i + 1
16: end while
17: while j ≤ r do
18: B.append(A[j])
19: j ← j + 1
20: end while
21: return B

```

## Algorithmus: Quicksort

Quicksort ist ein „Teile und Herrsche“-Algorithmus:

Die Daten werden in zwei Teile zerlegt und unabhängig voneinander sortiert.

Bei der Partitionierung muss sichergestellt werden, dass in der einen Teilmenge "kleinere" Elemente und in der anderen Teilmenge "größere" Elemente enthalten sind, um unabhängig sortieren zu können.

Dies geschieht durch (zufällige) Auswahl eines Pivot- Elements, das bei der Partitionierung als Vergleichswert für kleinere und größere Elemente herangezogen wird.

Komplexität:  $O(n^2)$  (Worst Case), Partitionierung in  $O(n)$

### QUICKSORT( $Z, l, r$ )

```

1: if $l < r$ then
2: $q \leftarrow \text{PARTITION}(Z, l, r)$
3: QUICKSORT($Z, l, q - 1$)
4: QUICKSORT($Z, q + 1, r$)
5: end if

```

### PARTITION( $Z, l, r$ )

```

1: $pivot \leftarrow Z[r]$
2: $i \leftarrow l$
3: $j \leftarrow r - 1$
4: while $i < j$ do
5: while $Z[i] < pivot$ do
6: $i \leftarrow i + 1$
7: end while
8: while $j > l$ and $Z[j] \geq pivot$ do
9: $j \leftarrow j - 1$
10: end while
11: if $i < j$ then
12: SWAP($Z[i], Z[j]$)
13: $i \leftarrow i + 1$
14: $j \leftarrow j - 1$
15: end if
16: end while
17: if $i = j$ and $Z[i] < pivot$ then
18: $i \leftarrow i + 1$
19: end if
20: if $Z[i] \neq pivot$ then
21: SWAP($Z[i], Z[r]$)
22: end if
23: return i

```

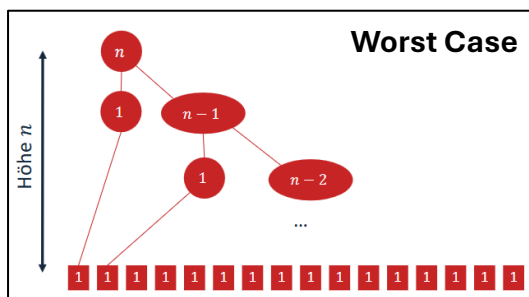
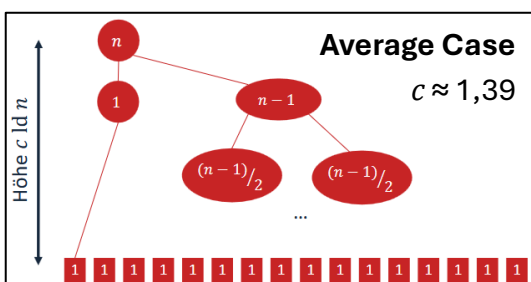
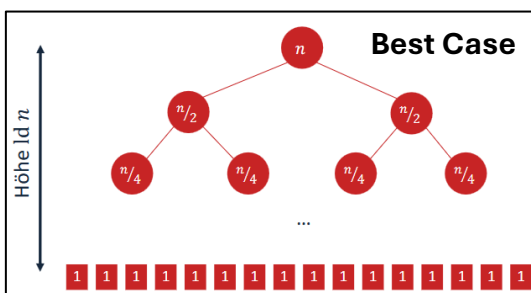
### Alternative Partitions-Funktion:

#### PARTITION2( $Z, l, r$ )

```

1: $pivot \leftarrow Z[r]$
2: $i \leftarrow l - 1$
3: for $j \leftarrow l$ to $r - 1$ do
4: if $Z[j] \leq pivot$ then
5: $i \leftarrow i + 1$
6: SWAP($Z[i], Z[j]$)
7: end if
8: end for
9: SWAP($Z[i + 1], Z[r]$)
10: return $i + 1$

```



## Algorithmus: Heapsort

Im unsortierten Array wird durch Vertauschungen die Max-Heap-Eigenschaft hergestellt (MAKE-MAX-HEAP), dass das Größte Element vorne (an der Wurzel) steht.

Das erste Element wird jetzt mit dem letzten Element getauscht und erneut die Max-Heap-Eigenschaft hergestellt (ohne die bereits sortierten Elemente am Ende). Die Sortierung findet also von hinten nach vorne statt.

Komplexität:  $O(n \log n)$ , Schleife (4-8) in  $O(n)$ , MAX-HEAPIFY in  $O(\log n)$

### HEAPSORT( $Z$ )

```

1: $n \leftarrow \text{length}(Z)$ ▷ Achtung: $Z[1 \dots n]$
2: MAKE-MAX-HEAP(Z)
3: $\text{heapsize} \leftarrow n$
4: for $i \leftarrow n$ downto 2 do
5: SWAP($Z[1], Z[i]$)
6: $\text{heapsize} \leftarrow \text{heapsize} - 1$
7: MAX-HEAPIFY($Z, 1, \text{heapsize}$)
8: end for
```

### MAKE-MAX-HEAP( $Z$ )

```

1: $n \leftarrow \text{length}(Z)$ ▷ Achtung: $Z[1 \dots n]$
2: for $i \leftarrow n/2$ downto 1 do
3: MAX-HEAPIFY(Z, i, n)
4: end for
```

### MAX-HEAPIFY( $Z, i, \text{heapsize}$ )

```

1: $l \leftarrow \text{LEFT}(i)$ ▷ Linker Kindknoten
2: $r \leftarrow \text{RIGHT}(i)$ ▷ Rechter Kindknoten
3: $\text{max} \leftarrow i$
4: if $l \leq \text{heapsize}$ and $Z[l] > Z[i]$ then
5: $\text{max} \leftarrow l$
6: end if
7: if $r \leq \text{heapsize}$ and $Z[r] > Z[\text{max}]$ then
8: $\text{max} \leftarrow r$
9: end if
10: if $\text{max} \neq i$ then
11: SWAP($Z[i], Z[\text{max}]$)
12: MAX-HEAPIFY($Z, \text{max}, \text{heapsize}$)
13: end if
```

## Definition Heap

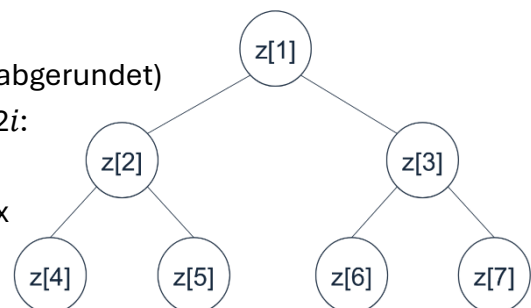
Ein **Heap** ist die Abbildung eines binären Baumes (jeder Knoten hat max. 2 Kinder) auf ein Array.

Die Wurzel des Baumes wird im ersten Element des Arrays abgelegt.

Die weiteren Knoten sind zeilenweise angeordnet.

### Regeln für die Navigation in einem Heap:

- Elternknoten von Knoten  $i$  hat den Index  $i/2$  (abgerundet)
- linke Kindknoten von Knoten  $i$  hat den Index  $2i$ :  
 $\text{LEFT}(i) \rightarrow 2i$
- rechte Kindknoten von Knoten  $i$  hat den Index  $2i+1$ :  
 $\text{RIGHT}(i) \rightarrow 2i+1$



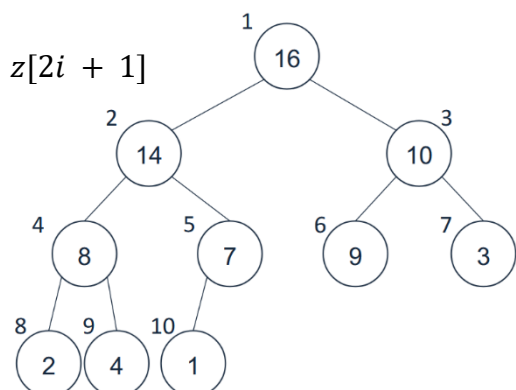
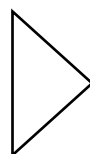
## Definition Max-Heap

Ein Array erfüllt die Max-Heap-Eigenschaft, wenn jeder Elternknoten größer oder gleich seinen Kindknoten ist.

Für jeden Knoten  $i$  gilt:  $z[i] \geq z[2i]$  und  $z[i] \geq z[2i + 1]$

### Array mit Max-Heap Eigenschaft

| 1  | 2  | 3  | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|----|----|----|---|---|---|---|---|---|----|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1  |



## Algorithmus: Counting-Sort

Notwendige **Bedingungen**, um den Counting-Sort anwenden zu können:

- Sortierkriterium sind positive ganze Zahlen
- Man kennt eine obere Schranke für die zu sortierenden Zahlen

Zuerst wird gezählt, wie oft jeder Wert vorkommt. Die Anzahl wird zu kumulierten Summen verarbeitet, welche die Position des Elements im Sortierten Array wiedergeben. Die Elemente werden dann an diese Positionen geschrieben.

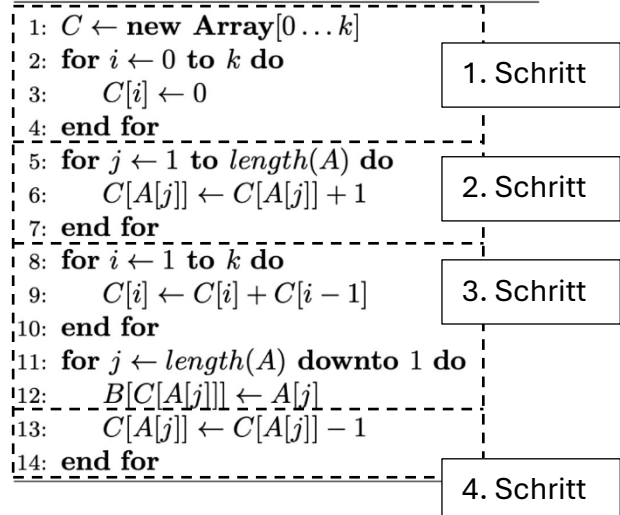
Komplexität:  $O(n + k)$  (oder  $O(n)$ )

### Beispielarray:

Maximum aller Zahlen:  $k = 5$

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| 2 | 5 | 3 | 0 | 2 | 3 | 0 | 3 |

### COUNTING-SORT( $A, B, k$ )



#### 1. Schritt: Initiale Befüllung des Hilfsarrays $C$

```

1: $C \leftarrow \text{new Array}[0 \dots k]$
2: for $i \leftarrow 0$ to k do
3: $C[i] \leftarrow 0$
4: end for

```

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |

#### 2. Schritt: Zählen der Häufigkeit der Werte

```

5: for $j \leftarrow 1$ to $\text{length}(A)$ do
6: $C[A[j]] \leftarrow C[A[j]] + 1$
7: end for

```

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 2 | 0 | 2 | 3 | 0 | 1 |

**ACHTUNG: Eingabe- und Ergebnisfeld beginnen bei Index 1!**

#### 3. Schritt: kumulierte Summe

```

8: for $i \leftarrow 1$ to k do
9: $C[i] \leftarrow C[i] + C[i - 1]$
10: end for

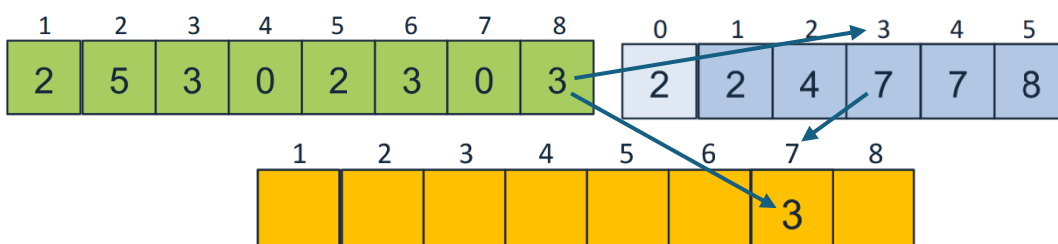
```

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 2 | 2 | 4 | 7 | 7 | 8 |

#### 4. Schritt: Ergebnisfeld Befüllen

Das Eingabefeld wird von hinten durchlaufen. Die Summe an der Stelle der Zahl im Hilfsarray, gibt die Position des Elements im Ergebnisfelds an.

Die Summe im Hilfsarray wird dann an dieser Stelle um 1 reduziert.



## Hashtabellen

- Das Speichern der Objekte vom Typ  $T$  erfolgt in einem Array der Länge  $m$ . Dieses Array wird auch Hashtabelle genannt.
- Die Festlegung, an welcher Stelle in der Hashtabelle ein Objekt abgelegt wird, ist durch eine Hashfunktion festgelegt:  $h: T \rightarrow \{0, \dots, m-1\}$

## Kollision

- Ist die Position, die von der Hashfunktion für ein neues Element berechnet wird, bereits belegt, nennt man das Kollision.
- Kollisionen sind nicht vermeidbar, denn:  
Die Mächtigkeit von  $T$  (Anzahl der möglichen zu speichernden Objekte) ist meist größer als  $m$ . Daher gilt:  $\exists o_1, o_2 \in T: o_1 \neq o_2 \wedge h(o_1) = h(o_2)$

## Wahl der Hashfunktion

- Die Hashfunktion  $h$  sollte **alle Plätze** der Hashtabelle **erreichen**. (Surjektivität)
- Alle Plätze der Hashtabelle sollten mit der **gleichen Wahrscheinlichkeit** erreicht werden. (Gleichverteilung)
- Die Hashfunktion sollte **effizient berechenbar** sein. (ideal:  $O(1)$ )
- **Ähnliche Objekte** sollten **nicht zu ähnlichen Hashwerten** führen. (Unstetigkeit)

## Hashfunktion: Modulo

*Für positive ganze Zahlen  $x$*

Eine gute Wahl von  $m$  ist entscheidend!

→  $h(x) = x \bmod m$  mit  $m$  als Primzahl

*Für Zeichenketten  $s$  der Länge  $n$*

Aufsummieren der ASCII- / UNICODE-Werte

→  $h(s) = (\sum_{i=0}^{n-1} \text{ascii}(s[i])) \bmod m$

**Probleme:**

- Anagramme:  $h(\text{„logarithmus“}) = h(\text{„algorithmus“})$
- Kleine Hashwerte bei kurzen Zeichenketten

*Verbesserte Hashfunktion*

→  $h(s) = (\sum_{i=0}^{n-1} \text{ascii}(s[i]) \cdot 256^{n-1-i}) \bmod m$

## Hashfunktion: Multiplikation

Sind die zu speichernden Objekte positive ganze Zahlen, wird das Objekt mit einer

irrationalen Zahl  $A$  multipliziert. („goldener Schnitt“ für  $A$ :  $A = \frac{\sqrt{5}-1}{2} \approx 0,6180339887 \dots$ )

Anschließend wird der gebrochene Anteil des Ergebnisses mit  $m$  multipliziert.

→  $h(x) = [(x \cdot A - \lfloor x \cdot A \rfloor) \cdot m]$



## Umgang mit Kollisionen

### Auflösung durch Verkettung

Die Hashtabelle enthält nicht die Objekte, sondern unsortierte verkettete Listen der Objekte.

#### Einfügen

Ein neues Element wird bei Kollision an den Anfang der Liste eingefügt.

Einfügen in  $O(1)$  möglich.

#### Suche

Die Suche eines Objektes setzt sich aus der Berechnung des Hashs und der Suche in der Liste zusammen. Die Mittlere Länge der Listen wird durch den

Belegungsfaktor  $\alpha = \frac{n}{m}$  mit  $n$  als Anzahl der Objekte und  $m$  als Länge der Hashtabelle Suche in  $O(1 + \alpha)$  möglich.

#### Löschen

Beim Löschen wird das Element gesucht, aus der Verketteten Liste entfernt und die Liste neu verkettet.

Löschen ist in  $O(1)$  möglich.

### Auflösung durch offene Adressierung

Die Hashfunktion liefert nicht eine Adresse, sondern eine Folge von Adressen, an denen das Objekt abgelegt werden kann.

Quadratisches sondieren:  $h(x, i) = (h'(x) + c_1i + c_2i^2) \bmod m$

$h'(x)$  ist die Hashfunktion,  $c_1$  und  $c_2$  sind Konstanten ungleich 0, die so gewählt sind, dass die Sondierungsfolge alle Slots erreicht.

Quadratisches sondieren vermeidet (primäre) Clusterbildung besser als lineare Sondierung.

Allerdings: Für zwei Schlüssel  $x_1$  und  $x_2$  mit  $h'(x_1) = h'(x_2)$  ergibt sich auch hier die gleiche Sondierungssequenz (sekundäre Clusterbildung).

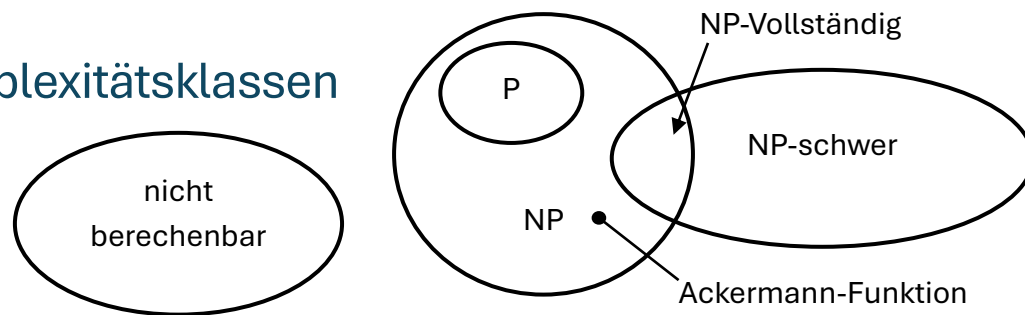
Bei einem Zugriff ist höchstens mit  $\frac{1}{1-\alpha}$  Sondierungen zu rechnen. (ideale Hashfunktion)

Ist  $\alpha$  hinreichend klein (in der Praxis  $\alpha < 0,75$ ) und konstant, ist Suchen, Einfügen und Löschen in  $O(1)$  möglich.

Beim Löschen kann ein Element nicht einfach entfernt werden, da eine Suche ggf. zu früh abgebrochen wird. Der Slot wird dann auf einen der Zustände gesetzt:

- **besetzt**
- **noch nie besetzt**
- **wieder frei**

## Komplexitätsklassen



### In polynomialer Zeit lösbare (effiziente) Probleme **P**

Algorithmen mit der Zeitkomplexität  $O(n^i)$  mit  $i \in \mathbb{N}_0$  gehören zur Klasse P. Diese Klasse beinhaltet alle Probleme, die in polynomialer Zeit lösbar sind.

### In polynomialer Zeit verifizierbare Probleme **NP**

Die Klasse NP (nichtdeterministisch polynomiell) enthält die Probleme, deren Lösung in polynomialer Zeit überprüft werden kann.

Beispiel: Ackermann-Funktion

$$A(0, n) = n + 1$$

$$A(m, 0) = A(m - 1, 1)$$

$$A(m, n) = A(m - 1, A(m, n - 1))$$

Selbst für kleine Werte von  $m$  und  $n$  kann die Berechnung von  $A(m, n)$  eine erhebliche Anzahl von Rechenschritten erfordern, die exponentiell in Bezug auf  $m$  und  $n$  wachsen.

### Schwierige NP-Probleme **NP-Vollständig**

Viele Probleme aus NP lassen sich auf andere Probleme in NP reduzieren.

Diese wenigen Probleme stellen die Schwierigsten Probleme von NP dar und werden als NP-Vollständig bezeichnet

Beispiele:

- Erfüllbarkeitsproblem (SAT)  
Eine bestimmte Belegung von  $n$  Variablen in einer booleschen Formel, dass die wahr ist.
- Traveling Salesman Problem (TSP)  
Bestimmung der kürzesten Rundreise durch  $n$  Städte.

### Nicht berechenbare Probleme

Es gibt Probleme, die mit einer Turing Maschine nicht berechnet werden können.

Beispiel: Halteproblem mit Programm  $X$  und Input  $y$

$$H(X, y) = \begin{cases} 0 & \text{wenn } X(y) \text{ terminiert} \\ 1 & \text{sonst} \end{cases}$$

| $Z(y)$                        |
|-------------------------------|
| 1: <b>repeat</b>              |
| 2: <b>until</b> $H(Z, y) = 1$ |

Falls  $Z$  beim Input  $y$  terminiert, liefert  $H(X, y)$  den Wert 0 und die Schleife wird wiederholt,  $Z$  terminiert nicht.

Falls  $Z$  aber beim Input  $y$  nicht terminiert, liefert  $H(X, y)$  den Wert 1 und das Programm endet. → **Widerspruch**